

GPT

■ 상위 작업	📌 <u>논문 세션 준비</u>
■ 진행 상태	<u>완료</u>
■ 첨부 파일	<u>Improving Language Understanding by Generative Pre-training.pdf</u>

0. Abstract

1. Introduction

2. Related Work

3. Method (제안 방법론)

3.1. Unsupervised pre-training

3.2. Supervised fine-tuning

3.3 Task-specific input transformations

4. Experiments (실험 및 결과)

Experiment1 (Unsupervised pre-training)

Experiment2 (Supervised Fine-tuning)

Analysis

5. Conclusion

0. Abstract

| 어떤 문제를 해결하려 했는가?

- Natural Language Understanding 을 위한 모델로, **Unlabeled Dataset** 을 활용할 수 있는 GPT를 제안한다
- 아래의 두 가지 방법을 결합함으로써 Unlabeled Dataset을 활용하여 더 좋은 모델 (GPT)을 얻는 것이 가능하다
 - **Generative pre-training** : Language Model 방식으로 Unlabeled text data를 학습

- Discriminative fine-tuning : 각 Downstream task 에 따라 fine-tuning 을 진행
- GPT는 fine-tuning 과정에서 모델 구조상의 최소한의 변형만 필요로 한다 (Add Linear Layer)
- Pre-training을 통해 일반화를 시도한 GPT는, 각 task에 맞게 변형되고 discriminative하게 학습된 모델들보다 우세한 성능을 보였다. 벤치마크 12개 중 9개에서 유의미한 성능 우세를 보였다.

1. Introduction

| 연구 동기와 문제점은 무엇인가?

0) 연구 동기

- 대부분의 딥러닝 NLP 모델은 많은 양의 라벨 데이터가 필요하나 실제로는 부족하다.
- 비라벨 데이터는 풍부하므로 이를 활용해 범용 표현을 학습하는 것이 필요하다.
→ GPT는 모델 구조가 아닌 데이터 구조의 한계에 의해 등장한 모델

1) 논문이 다루는 task

- Input: 자연어 텍스트 (문장/문단/질문-답변 쌍 등)
- Output: NLI(참/거짓/중립), QA(정답 선택), 문장 유사도(동일/비동일), 분류(긍정/부정, 문법적/비문법적)

2) 해당 task에서 기존 연구 한계점

- **Unlabeled Dataset을 Unsupervised 방식으로** 사전학습에 활용했을 때 (Semi-supervised Learning), 더 좋은 representation을 얻을 수 있다는 것이 선행연구에서 보였다.
- 하지만 이러한 **Semi-supervised Learning 방식의 접근에는 2가지 어려움**이 있다.
 - **학습 목표 설정**
: 어떤 pre-train objective(최적화 목표)가 transfer-learning에 최적일지 불명확하다.

- 후보 연구:
 - 언어모델링 (Language Modeling, LM): 단어 분포/언어 구조 포착에 강점
 - 기계번역 (Machine Translation, MT): 의미 대응 좋지만 범용성 한계
 - 담화 일관성 (Discourse Coherence, DC): 긴 문맥 이해 가능하지만 태스크 제한적

→ 다양한 후보가 있었으나, GPT는 LM을 선택

◦ transfer 방식 문제

: 사전학습된 모델을 각각 다운스트림 태스크에 적용할 때 어떤 방법을 통해야 효율적일지

- task별 구조 변경 필요 → 범용성 저하
- 데이터 적은 task에서는 성능 저하 발생
- 효율적인 fine-tuning 전략 정의 어려움

Traversal-Style Input Transformation

→ 이전 연구들은 Task 맞춤형 모델을 만들어 모델을 고정한 채 아키텍처, 학습방법, objective를 task마다 조정했다면, GPT-1은 하나의 언어모델(LM)을 사전학습으로 만들어두고 각 downstream task를 입력 포맷으로 바꿔주는 방식으로 통일해서 적용함

2. Related Work

| 관련 연구 동향은 어떠한가?

• Semi-supervised Learning

- 초기 연구는 unlabeled data를 활용해 word-level 통계치나 embeddings를 만들어 성능을 개선했으나, transfer되는 정보가 단어 수준에 머물러 한계가 있었다.
 - 예시: **Word2Vec (2013)** 은 단어를 벡터로 표현해 유사도와 관계를 설명할 수 있었으나, 문맥적 의미는 반영하지 못했다.
 - GPT는 단어를 넘어 **semantic-level context**를 학습하고 transfer함으로써 더 높은 수준의 의미 이해를 가능하게 했다.

- **Unsupervised Pre-training**

- 모델의 initialization을 잘 설정하기 위한 방법으로 발전해왔으며, LSTM 기반 language model pre-training은 존재했지만 long-term dependency를 처리하는 데 한계가 있었다.

- 예시: **Seq2Seq + Attention (2014~15)** 은 인코더-디코더 구조와 Attention을 통해 긴 문장 처리를 개선했으나 RNN 구조 한계가 있었다.
- 예시: 이후 **Transformer (2017)** 가 등장하며 RNN을 제거하고 Self-Attention으로 병렬 학습과 장기 문맥 처리가 가능해졌다.
- GPT는 **Transformer 아키텍처**를 사용하여 긴 문맥을 효과적으로 학습하고, transfer 시에도 구조적 수정이 거의 필요 없는 범용성을 보여주었다.

- **Auxiliary Objective**

- language modeling과 같은 auxiliary objective를 추가하면 모델의 generalization과 convergence 속도를 높일 수 있음이 여러 연구에서 입증되었다.
- 예시: **ELMo (2018)** 는 language modeling을 활용한 contextual embedding을 제시해 단어 의미를 문맥에 따라 달리 학습할 수 있었지만, 여전히 task-specific 구조 조정이 필요했다.
- GPT 역시 fine-tuning 과정에서 이 방식을 사용하지만, 이미 pre-training 단계에서 target task와 관련한 언어정보를 충분히 학습했다는 점에서 차별성이 있다.

3. Method (제안 방법론)

| 연구 접근법과 모델 구조는 어떻게 되는가?

Main Idea

→ "Unsupervised pre-training + Supervised fine-tuning으로 범용 NLU 모델을 만들자."

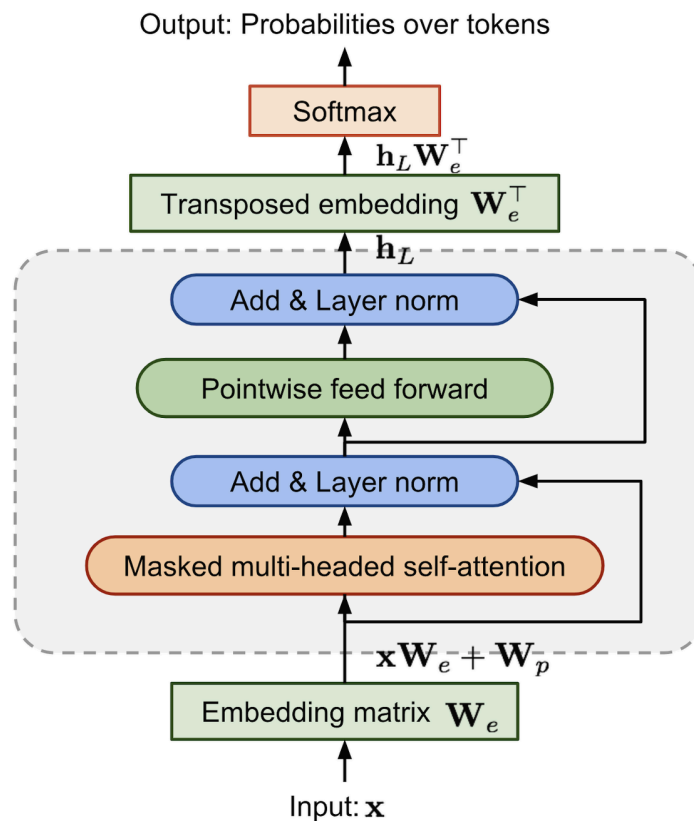
→ Large corpus + Language Modeling → Labeled Data + Fine-tuning

3.1. Unsupervised pre-training

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta)$$

Objective of Pre-train : Language Modeling

- Pre-training LM loss
 - **역할**: 라벨 없는 대규모 corpus로 언어모델을 학습하는 단계이다.
 - **목표**: 앞 단어들을 보고 **다음 단어(token)** 가 나올 확률을 최대화하여, 범용 언어 지식과 문맥 표현을 습득한다.
 - 앞의 k개 단어 맥락을 보고 실제 단어 u_i 가 나올 확률의 log-likelihood를 합산하는 방식으로, 주어진 문맥에서 실제로 나타난 단어가 모델이 예측한 확률 분포에서 높은 확률을 가지도록 파라미터 Θ 를 학습하는 과정이다.

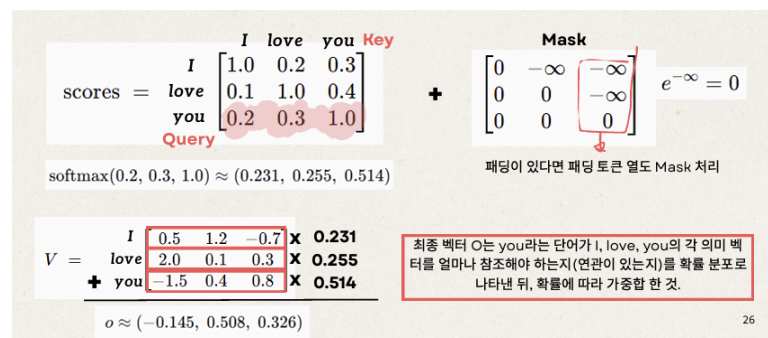


$$h_0 = UW_e + W_p$$

$$h_l = \text{transformer_block}(h_{l-1}) \forall l \in [1, n]$$

$$P(u) = \text{softmax}(h_n W_e^T)$$

- **입력 (Input):** 토큰을 embedding matrix W_e 로 변환하고, positional embedding W_p 를 더해 문맥+위치 정보 포함된 벡터 h_0 생성한다.
- **Transformer Decoder Blocks (12층):**
 - 각 block은 **Masked Multi-Head Self-Attention** → **Add & LayerNorm** → **Feed Forward** → **Add & LayerNorm** 순서로 구성된다.
 - Masked self-attention을 사용해 **앞 단어만 참고**하여 다음 단어 예측 가능하게 한다.



- **출력 (Output):** 마지막 hidden state h_n 을 embedding matrix W_e^T 와 곱해 vocabulary 차원으로 변환 후, softmax로 단어 분포를 예측한다.

1. **Masked Multi-head Self-Attention**으로 앞 단어만 참고 → autoregressive LM.
2. 긴 문맥을 효과적으로 학습 가능 (attention은 모든 이전 토큰 참조).
3. 병렬 연산 가능 → 학습 효율성 ↑.
4. downstream task에선 **task-specific head**만 추가하고 본체 구조 수정 거의 없음.

▼ Transformer Block 기본 순서(복습)

- 입력 임베딩 ($h_0 = UW_e + W_p$)

- **Masked Multi-Head Self-Attention**
 - Q/K/V → Score 계산
 - Softmax로 Attention Weights 산출
 - Value 결합
- **잔차 연결 + LayerNorm (Norm-1)**
 - $H_1 = LN(h_0 + O)$
- **Feed Forward Network (FFN)**
 - 각 토큰에 독립적으로 2-layer MLP 적용
 - $FFN(H_1) = W_2\phi(W_1H_1 + b_1) + b_2$
 - 차원 확장(768 → 3072 → 768)
- **잔차 연결 + LayerNorm (Norm-2)**
- **출력** (다음 블록 입력으로 전달)

3.2. Supervised fine-tuning

$$L_2(\mathcal{C}) = \sum_{(x,y)} \log P(y|x^1, \dots, x^m).$$

Auxiliary Objective : Maximize log probability to predict y with given x

- target task loss
 - **역할**: labeled dataset을 활용하여 downstream task (분류, QA, NLI 등)를 학습하는 단계이다.
 - **목표**: 입력 시퀀스가 주어졌을 때, 올바른 **정답 라벨** y 의 확률을 최대화한다.
 - 입력 시퀀스 $(x^1, \dots, x^m)$ 이 주어졌을 때, 정답 라벨 y 가 나올 확률의 **log-likelihood**를 합산한다.
 - 사전학습된 모델이 **task-specific output layer**를 통해 정답 분포를 잘 맞추도록 조정한다.
 - 위에서 사전학습된 언어모델에 추가된 linear layer W_y 를 fine-tuning하여, 각 태스크의 정답 예측 확률을 최대화한다.

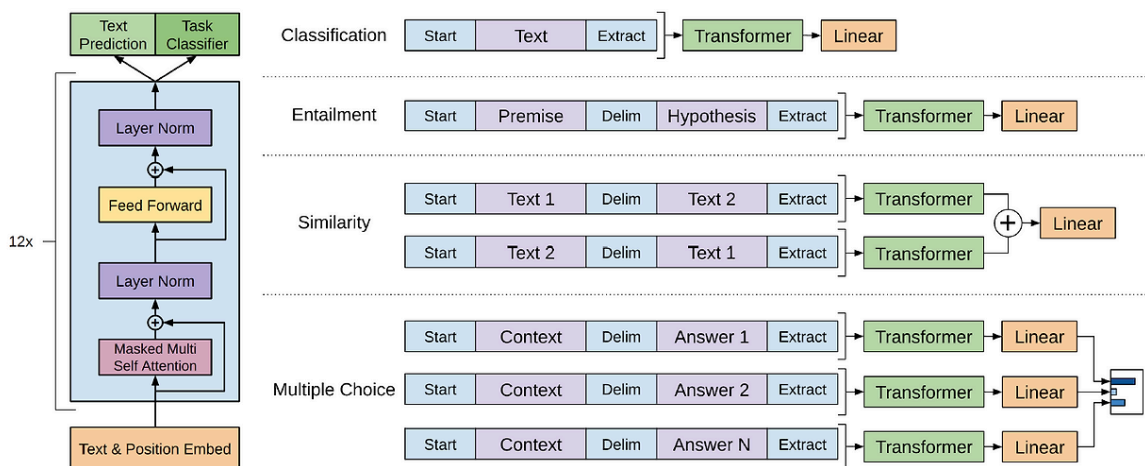
$$P(y|x^1, \dots, x^m) = \text{softmax}(h_l^m W_y).$$

- 이때 $L_2(\mathcal{C})$ 내부적으로 마지막 hidden state에 linear layer를 곱해 라벨별 확률 분포를 계산하고, 정답 라벨의 확률을 최대화하는 방식으로 동작한다.

$$L_3(\mathcal{C}) = L_2(\mathcal{C}) + \lambda * L_1(\mathcal{C})$$

- 최종 L_3 에서는 이 supervised loss L_2 에 pre-training loss L_1 가 더해져 fine-tuning 성능과 안정성을 동시에 확보한다.

3.3 Task-specific input transformations



- GPT는 모델 구조를 바꾸지 않고 입력을 변환해 downstream tasks에 적용한다.
- [Start], [Delimiter], [Extract] 같은 특별 토큰을 활용해 태스크를 시퀀스 문제로 변환한다.
- 이 방식은 단일 아키텍처로 다수의 NLP task 처리를 가능하게 만든 중요한 기여다.

▼ +) 왜 Decoder만 썼는가?

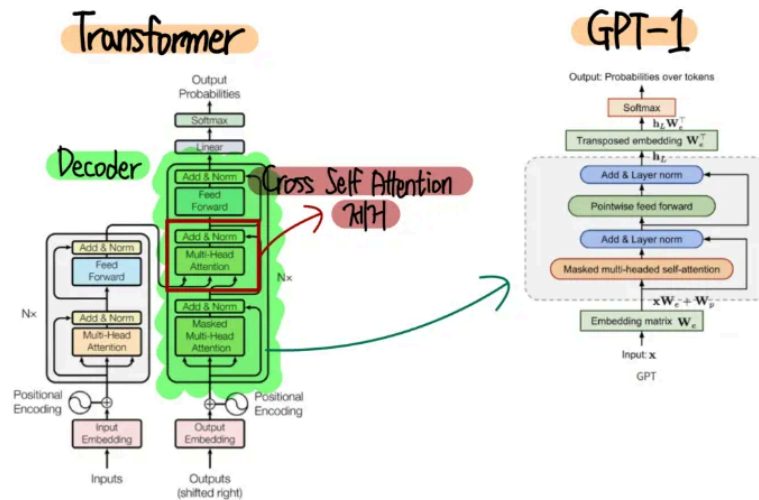


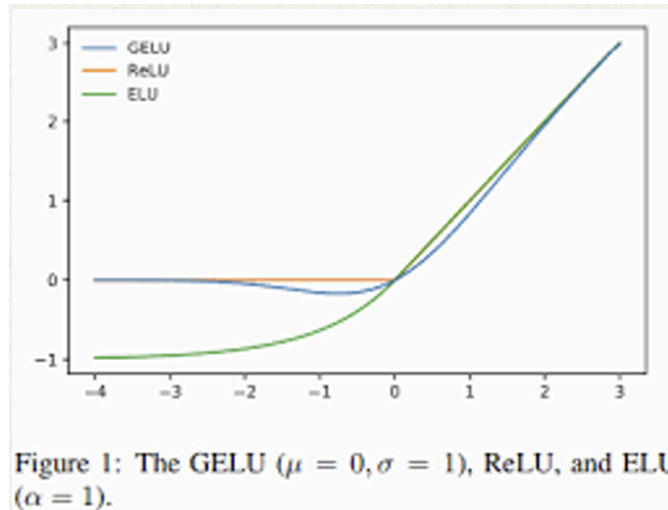
Figure 1: The Transformer - model architecture.

- GPT는 언어모델링이 Objective이기 때문!
- 언어모델은 앞 단어를 보고 다음 단어 예측하는 Autoregressive 구조가 자연스럽기 때문.
- 이 구조는 Decoder의 Masked Self-Attention과 정확히 맞아떨어짐.
- Encoder는 필요 없으니 뺀 것 → 따라서 Cross-Attention도 함께 사라짐.

4. Experiments (실험 및 결과)

| 실험 구성과 결과는?

Experiment1 (Unsupervised pre-training)



- 데이터셋 : BookCorpus (7천 권, 8억 단어)
- 모델: Transformer decoder 12층
- 기법: BPE 토크나이저, Adam optimizer, GELU activation
- 결론: BookCorpus에서 학습된 LM이 범용적 언어 표현을 잘 배운다.

Experiment2 (Supervised Fine-tuning)

Method	Classification		Semantic Similarity			GLUE
	CoLA (mc)	SST2 (acc)	MRPC (F1)	STS-B (pc)	QQP (F1)	
Sparse byte mLSTM [16]	-	93.2	-	-	-	-
TF-KLD [23]	-	-	86.0	-	-	-
ECNU (mixed ensemble) [60]	-	-	-	<u>81.0</u>	-	-
Single-task BiLSTM + ELMo + Attn [64]	35.0	90.2	80.2	55.5	66.1	64.8
Multi-task BiLSTM + ELMo + Attn [64]	18.9	91.6	83.5	72.8	63.3	<u>68.9</u>
Finetuned Transformer LM (ours)	45.4	91.3	82.3	82.0	70.3	72.8

Method	Story Cloze	RACE-m	RACE-h	RACE
val-LS-skip [55]	76.5	-	-	-
Hidden Coherence Model [7]	<u>77.6</u>	-	-	-
Dynamic Fusion Net [67] (9x)	-	55.6	49.4	51.2
BiAttention MRU [59] (9x)	-	<u>60.2</u>	<u>50.3</u>	<u>53.3</u>
Finetuned Transformer LM (ours)	86.5	62.9	57.4	59.0

Method	MNLI-m	MNLI-mm	SNLI	SciTail	QNLI	RTE
ESIM + ELMo [44] (5x)	-	-	<u>89.3</u>	-	-	-
CAFE [58] (5x)	80.2	79.0	<u>89.3</u>	-	-	-
Stochastic Answer Network [35] (3x)	<u>80.6</u>	<u>80.1</u>	-	-	-	-
CAFE [58]	78.7	77.9	88.5	<u>83.3</u>	-	-
GenSen [64]	71.4	71.3	-	-	<u>82.3</u>	59.2
Multi-task BiLSTM + Attn [64]	72.2	72.1	-	-	82.1	61.7
Finetuned Transformer LM (ours)	82.1	81.4	89.9	88.3	88.1	56.0

- 데이터셋: 12개 벤치마크 (NLI, QA, 문장 유사도, 분류)

Table 1: A list of the different tasks and datasets used in our experiments.

Task	Datasets
Natural language inference	SNLI [5], MultiNLI [66], Question NLI [64], RTE [4], SciTail [25]
Question Answering	RACE [30], Story Cloze [40]
Sentence similarity	MSR Paraphrase Corpus [14], Quora Question Pairs [9], STS Benchmark [6]
Classification	Stanford Sentiment Treebank-2 [54], CoLA [65]

Benchmark Datasets of GPT-1 (Task : NLI, QA, Semantic Similarity, Classification)

- **모델:** Pre-trained LM 위에 **linear layer 추가** 후 task-specific fine-tuning
- **기법:** Supervised loss L_2 + Pre-training loss L_1 결합 ($L_3 = L_2 + \lambda L_1$)
- **결과:** 12개 벤치마크 중 9개에서 SOTA 달성
 - MultiNLI: 82.1%, SNLI: 89.9%, SciTail: 88.3%
 - RACE: 59.0% (+8~9% 향상), Story Cloze: 86.5% (기존 대비 +9%)
 - SST-2: 91.3%, CoLA: 45.4 (Matthews corr.)
- **결론:** 단순한 구조 수정만으로도 다양한 태스크에서 범용적 성능을 달성하였으며, 특히 소규모 데이터 태스크에서 큰 성능 향상을 보였다.

Analysis

- **Impact of number of layers transferred**

레이어를 많이 전이할수록 fine-tuning 성능이 좋아진다.

- **Zero-shot Behaviors**

fine-tuning 없이도 zero-shot 성능을 보이며, LSTM보다 Transformer가 전이에 유리하다.

- **Ablation Studies**

ablation 결과, pre-training과 auxiliary LM이 성능에 필수적임이 확인된다.

5. Conclusion

| 결론 및 주요 요약은?

- generative pre-training + discriminative fine-tuning이 효과적이라는 것을 보였다.

- 특히, 많은 양의 연속적인 텍스트를 학습하는 것이 세상에 대한 지식과 장기 의존성을 처리하는 데 필요한 능력을 얻게 하고,
- discriminative task로 그 지식이 잘 transfer되었을 때 효과적임을 알 수 있었다.
- 이러한 큰 성능 향상이 실제로 실현 가능하다는 것을 이 연구를 통해 보였고, 이를 위해서는 transformer와 장기 의존성을 가진 텍스트가 가장 적합함을 보였다.